



KTU NOTES

The learning companion.

**KTU STUDY MATERIALS | SYLLABUS | LIVE
NOTIFICATIONS | SOLVED QUESTION PAPERS**

Website: www.ktunotes.in

MODULE-4

SYLLABUS

Instruction set - Instruction types - Addressing modes of 8051 microcontrollers.

8051 microcontroller data types and directives - Time delay programmes and I/O port programming.

Introduction to embedded C Programming - time delay in C - I/O port programming in embedded C.

Ktunotes.in

The instructions of 8051 can be broadly classified under the following headings.

1. Data transfer instructions
2. Arithmetic instructions
3. Logical instructions
4. Branch instructions
5. Subroutine instructions
6. Bit manipulation instructions

1. Data transfer instructions.

In this group, the instructions perform data transfer operations of the following types.

- a. Move the contents of a register Rn to A
 - i. `MOV A,R2`
- b. Move the contents of a register A to Rn
 - i. `MOV R4,A`
- c. Move an immediate 8 bit data to register A or to Rn or to a memory location(direct or indirect)
 - i. `MOV A, #45H` (Move value 45 to A)
 - ii. `MOV R6, #51H` (Move value 51 to R6)
 - iii. `MOV 30H, #44H` (Move value 44 to RAM location 30)
 - iv. `MOV @R0, #0E8` (Move value E8 to RAM location addressed by R0)
 - v. `MOV DPTR, #0F5A2H` (Move value F5A2 to DPTR)
- d. Move the contents of a memory location to A or A to a memory location using direct and indirect addressing
 - i. `MOV A, 65H` (Move data at RAM location 65 to A)
 - ii. `MOV A, @R0` (Move data at RAM location addressed by R0 to A)
 - iii. `MOV 45H, A` (Move data at A to RAM location 45)
 - iv. `MOV @R1, A` (Move data at A to RAM location addressed by R1)
- e. Move the contents of a memory location to Rn or Rn to a memory location using direct addressing
 - i. `MOV R3, 65H` (Move data at RAM location 65 to R3)
 - ii. `MOV 45H, R2` (Move data at R2 to RAM location 45)
- f. Move the contents of memory location to another memory location using direct and indirect addressing
 - i. `MOV 47H, 65H` (Move data at RAM location 65 to RAM location 47)
 - ii. `MOV 45H, @R0` (Move data at RAM location addressed by R0 to RAM location 45)
- g. Move the contents of an external memory to A or A to an external memory
 - i. `MOVX A,@DPTR` (Move data at external RAM location addressed by DPTR to A)
 - ii. `MOVX @DPTR,A`
- h. Move the contents of program memory to A
 - i. `MOVC A, @A+PC`
 - ii. `MOVC A, @A+DPTR`

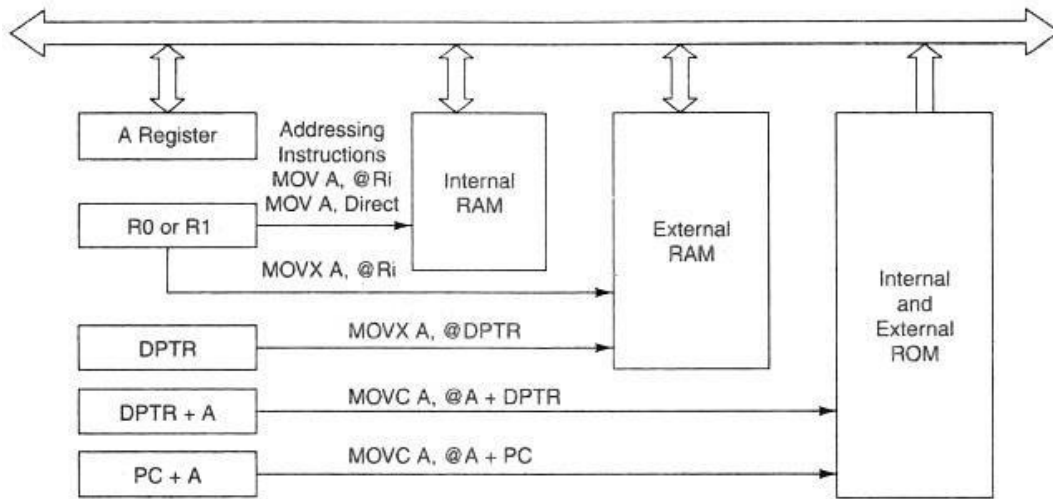


FIG. Addressing Using MOV, MOVX and MOVC

i. Push and Pop instructions

PUSH 6 Increment SP ($SP=SP+1$) then, data at RAM location 06 is moved to stack memory addressed by SP.

POP 7 Data at stack memory addressed by SP is moved to RAM location 07 then decrement SP ($SP=SP-1$).

j. Exchange instructions

The content of source ie, register, direct memory or indirect memory will be exchanged with the contents of destination ie, accumulator.

- i. XCH A,R3 Exchange the data at A and register R3
- ii. XCH A,@R1 Exchange the data at A and RAM location addressed by R1
- iii. XCH A,54h Exchange the data at A and RAM location 54

k. Exchange digit. Exchange the lower order nibble of Accumulator (A0-A3) with lower order nibble of the internal RAM location which is indirectly addressed by the register.

- i. XCHD A,@R1
- ii. XCHD A,@R0

2. Arithmetic instructions.

The 8051 can perform addition, subtraction, multiplication and division operations on 8 bit numbers.

Addition

In this group, we have instructions to

- i. Add the contents of A with immediate data with or without carry.
 - i. ADD A, #45H (A = A + 45)
 - ii. ADDC A, #0B4H (A = A + 45 + CY Flag)
- ii. Add the contents of A with register Rn with or without carry.
 - i. ADD A, R5 (A = A + Data at R5)
 - ii. ADDC A, R2 (A = A + Data at R2 + CY Flag)
- iii. Add the contents of A with contents of memory with or without carry using direct and indirect addressing
 - i. ADD A, 51H (A = A + Data at RAM location 51)
 - ii. ADDC A, 75H (A = A + Data at RAM location 75 + CY Flag)
 - iii. ADD A, @R1 (A = A + Data at RAM location addressed by R1)
 - iv. ADDC A, @R0 (A = A + Data at RAM location addressed by R0 + CY Flag)

Subtraction

In this group, we have instructions to

- i. Subtract the contents of A with immediate data with carry.

SUBB A, #45H (A = A - 45 - CY Flag)
- ii. Subtract the contents of A with register Rn with carry.

SUBB A, R5 (A = A - Data at R2 - CY Flag)
- i. Subtract the contents of A with contents of memory with or without carry using direct and indirect addressing
 - i. SUBB A, 51H (A = A + Data at RAM location 51 - CY Flag)
 - ii. SUBB A, @R1 (A = A - Data at RAM location addressed by R1 - CY Flag)

Multiplication

MUL A B.

- This instruction multiplies two 8 bit unsigned numbers which are stored in A and B registers.
- After multiplication the lower byte of the result will be stored in accumulator and higher byte of result will be stored in B register.

Eg. MOV A, #45H ;[A]=45H
 MOV B, #0F5H ;[B]=F5H
 MUL AB ;[A] x [B] = 45 x F5 = 4209
 ;[A]=09H, [B]=42H

Division

DIV A B.

- This instruction divides the 8 bit unsigned number which is stored in A by the 8 bit unsigned number which is stored in B register.
- After division the result will be stored in accumulator and remainder will be stored in B register.

Eg. MOV A,#45H ;[A]=0E8H
 MOV B,#0F5H ;[B]=1BH
 DIV AB ;[A] / [B] = E8 / 1B = 08 H with remainder 10H
 ;[A] = 08H, [B]=10H

DA A (Decimal Adjust After Addition).

When two BCD numbers are added, the answer is a non-BCD number sometimes. To get the result in BCD, we use DA A instruction after the addition. DA A works as follows.

- If lower nibble is greater than 9 or auxiliary carry is 1, 6 is added to lower nibble.
- If upper nibble is greater than 9 or carry is 1, 6 is added to upper nibble.

Increment: increments the operand by one.

- INC increments the value of source by 1.
- If the initial value of register is FFh, incrementing the value will cause it to reset to 0.

INC A A=A+1

INC Rn Eg; INC R4; R4= R4+1

INC DIRECT Eg; INC 54; RAM location 54= RAM location 54 +1

INC @Ri Eg; INC @R3; RAM location addressed by R3 = RAM location addressed by R3 +1

INC DPTR DPTR=DPTR+1

Decrement: decrements the operand by one.

- DEC decrements the value of *source* by 1.
- If the initial value of is 0, decrementing the value will cause it to reset to FFh.

DEC A A=A-1

DEC Rn Eg; DEC R4; R4= R4-1

DEC DIRECT Eg; DEC54; RAM location 54= RAM location 54 -1

DEC @Ri Eg; DEC @R3; RAM location addressed by R3 = RAM location addressed by R3 -1

3. Logical Instructions

Logical AND

ANL destination, source:

- **ANL** does a bitwise "AND" operation between *source* and *destination*, leaving the resulting value in *destination*.

ANL A,#DATA	eg:- ANL A,#09 (A = A and value 09)
ANL A, Rn	eg:- ANL A,R1 (A = A and Data at R1)
ANLA, DIRECT	eg:- ANL A,46 (A = A and Data at RAM location 41)
ANL A,@Ri	eg:- ANL A,@R4 (A = A and Data at RAM location addressed by R4)
ANL DIRECT,A	eg:- ANL 45,A (RAM location 45 = RAM location 45 and A)
ANL DIRECT, #DATA	eg:- ANL 45,#FF (RAM location 45 = RAM location 45 and value FF)

Logical OR

- **ORL** destination, source: **ORL** does a bitwise "OR" operation between *source* and *destination*, leaving the resulting value in *destination*.

ORL A,#DATA	eg:- ORL A,#09 (A = A or value 09)
ORL A, Rn	eg:- ORL A,R1 (A = A or Data at R1)
ORL A,DIRECT	eg:- ORL A,46 (A = A or Data at RAM location 41)
ORL A,@Ri	eg:- ORL A,@R4 (A = A or Data at RAM location addressed by R4)
ORL DIRECT,A	eg:- ORL 45,A (RAM location 45 = RAM location 45 or A)
ORL DIRECT, #DATA	eg:- ORL 45,#FF (RAM location 45 = RAM location 45 or value FF)

Logical Ex-OR

- **XRL** destination, source: **XRL** does a bitwise "EX-OR" operation between *source* and *destination*, leaving the resulting value in *destination*.

XRL A,#DATA	eg:- XRL A,#09 (A = A xor value 09)
XRL A,Rn	eg:- XRL A,R1 (A = A xor Data at R1)
XRL A,DIRECT	eg:- XRL A,46 (A = A xor Data at RAM location 41)
XRL A,@Ri	eg:- XRL A,@R4 (A = A xor Data at RAM location addressed by R4)
XRL DIRECT,A	eg:- XRL 45,A (RAM location 45 = RAM location 45 xor A)
XRL DIRECT, #DATA	eg:- XRL 45,#FF (RAM location 45 = RAM location 45 xor value FF)

Logical NOT

- **CPL** complements *operand*, leaving the result in *operand*. If *operand* is a single bit then the state of the bit will be reversed.

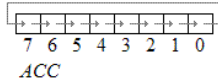
CPL A	Invert all the bits of A.
CPL C	Invert Carry flag.
CPL bit address	Eg:- CPL 07 07 is the address of 8 th bit of first bit addressable memory location .
Invert it.	

SWAP A – Swap the upper nibble and lower nibble of A

Rotate Instructions

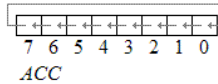
RR A

This instruction is rotate right the accumulator. Its operation is illustrated below. Each bit is shifted one location to the right, with bit 0 going to bit 7.



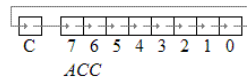
RL A

Rotate left the accumulator. Each bit is shifted one location to the left, with bit 7 going to bit 0



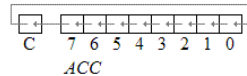
RRCA

Rotate right through the carry. Each bit is shifted one location to the right, with bit 0 going into the carry bit in the PSW, while the carry was at goes into bit 7



RLCA

Rotate left through the carry. Each bit is shifted one location to the left, with bit 7 going into the carry bit in the PSW, while the carry goes into bit 0.

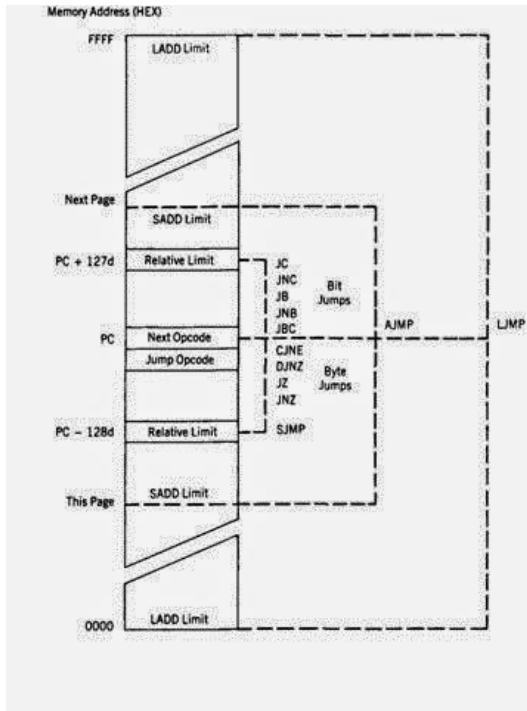


4. **Branch (JUMP) Instructions**

- A jump or call instruction can replace the contents of program counter with a new address given in the instruction that causes the program execution to begin at the code stored at the new address.

There are 3 types of jump instructions. They are:-

1. Relative Jump
2. Short Absolute Jump
3. Long Absolute Jump



Jump instruction ranges

1. *Relative Jump*

Jump allows only to a range of bytes within +127d or -128d from the instruction following jump.

Relative Jump Instructions:

a. *Unconditional jump:*

SJMP <relative address> ; Jump to the address specified unconditionally. Jump allows only to relative range.

b. *Conditional jump:*

• Bit level Jump Instructions:

JC <relative address> Jump if CY = 1

JNC <relative address> Jump if CY = 0

JB bit, <relative address>

Eg; JB 08, <relative address>

Jump to the relative address specified if the bit with address 08 = 1

JNB bit, <relative address>

Eg; JNB 08, <relative address>

Jump to the relative address specified if the bit with address 08 = 0

JBC bit, <relative address>

jump to the relative address specified if the direct bit is set and then clear the bit.

- Byte level Jump Instructions:

CJNE <destination byte>, <source byte>, <relative address>

Compare the two bytes of data(destination byte and source byte) and if these are not equal then jump to the relative address specified.

For Eg: CJNE A,56, <relative address>

Compare the data at A and data at RAM location 56 and if these are not equal then jump to the relative address specified.

DJNZ Rn <relative address>

Decrement the data at register Rn and jump to the relative address specified if the decremented value not equal to 0.

JZ <relative address> Jump to the relative address specified if data at A equal to 0.

JNZ <relative address> Jump to the relative address specified if data at A not equal to 0.

2. *Short Absolute Jump*

- In 8051, 64 K byte of program memory space is divided into 32 pages of 2K byte each.
- Jump allows only to a range of bytes within the 2K byte from the instruction following jump.

Example of short absolute jump: -

AJMP <address> ; Jump to the address specified unconditionally. Jump allows only to 2K range.

3. *Long Absolute Jump*

- Applications that need to access the entire program memory from 0000H to FFFFH use long absolute jump.
 - Jump allows to any locations among the 64 K memory.
- LJMP <address> ; Jump to the address specified unconditionally. Jump allows to anywhere.

5. **Subroutine CALL And RETURN Instructions**

- Subroutines are subprograms used to simplify the programs whenever a particular task is to be done repeatedly.
- Subroutines are handled by CALL and RET instructions

There are two types of CALL instructions

1. **LCALL address (16 bit)**

This is long call instruction which unconditionally calls the subroutine located at the indicated 16 bit address. Call the subroutine located anywhere in the program memory space. Push the address of instruction immediately after the LCALL instruction on to the stack.

2. **ACALL address**

This is absolute call instruction which unconditionally calls the subroutine located at the specified address. Call the subroutine located on the same page. Push the address of instruction immediately after the LCALL instruction on to the stack.

RET instruction

RET instruction pops top two contents from the stack and load it to PC

6. Bit Manipulation Instructions.

- 8051 has 128 bit addressable memory, bit addressable SFRs and bit addressable PORT pins.
- It is possible to perform following bit wise operations for these bit addressable locations.

1. LOGICAL AND
 - a. ANL C,BIT(BIT ADDRESS) ; 'LOGICALLY AND' CARRY AND CONTENT OF BIT ADDRESS, STORE RESULT IN CARRY
 - b. ANL C, /BIT; ; 'LOGICALLY AND' CARRY AND COMPLEMENT OF CONTENT OF BIT ADDRESS, STORE RESULT IN CARRY
2. LOGICAL OR
 - a. ORL C,BIT(BIT ADDRESS) ; 'LOGICALLY OR' CARRY AND CONTENT OF BIT ADDRESS, STORE RESULT IN CARRY
 - b. ORL C, /BIT; ; 'LOGICALLY OR' CARRY AND COMPLEMENT OF CONTENT OF BIT ADDRESS, STORE RESULT IN CARRY
3. CLR bit
 - a. CLR bit ; CONTENT OF BIT ADDRESS SPECIFIED WILL BE CLEARED.
 - b. CLR C ; CONTENT OF CARRY WILL BE CLEARED.
4. CPL bit
 - a. CPL bit ; CONTENT OF BIT ADDRESS SPECIFIED WILL BE COMPLEMENTED.
 - b. CPL C ; CONTENT OF CARRY WILL BE COMPLEMENTED.

4.2 ADDRESSING MODES

- The method of specifying the data to be operated by the instruction is called addressing
- 8051 addressing modes are classified as follows.

1. Immediate addressing.
2. Register addressing.
3. Direct addressing.
4. Indirect addressing.
5. Indexed addressing

1. Immediate addressing.

- In this addressing mode the data is provided as a part of instruction itself.
- In other words data immediately follows the instruction.

Eg. MOV A,#30H It means that immediate data 30h provided in instruction is moved into A register.
ADD A, #83 It means that immediate data 30h provided in instruction is added with A register.

Symbol indicates the data is immediate.

2. Register addressing.

- In this addressing mode the register will hold the data.
- One of the eight general registers (R0 to R7) can be used and specified as the operand.

Eg. MOV A,R0 Content of R0 register is copied into Accumulator.
ADD A,R6 Content of R0 register is added with Accumulator.

R0 – R7 will be selected from the current selection of register bank. The default register bank will be bank 0.

3. Direct addressing

- In this mode the direct address of memory location is provided in instruction to fetch the operand.
- Only internal RAM and SFR's address can be used in this type of instruction.
Ex: MOV A, 30H => Content of RAM address 30H is copied into Accumulator.

4. Indirect addressing

- Here the address of memory location is indirectly provided by a register.
- The '@' sign indicates that the register holds the address of memory location i.e. fetch the content of memory location whose address is stored in register.
Ex: MOV A,@R0 => Copy the content of memory location whose address is given in R0 register.

5. Indexed addressing

- This addressing mode is basically used for accessing data from look up table.
- Here the address of memory is indexed i.e. added to form the actual address of memory.

Ex: MOVC A,@A+DPTR => here 'C' means Code. Here the content of A register is added with content of DPTR and the resultant is the address of memory location from where the data is copied to A register.

4.3 DATA TYPES AND DIRECTIVES

DATA TYPES IN ASSEMBLY LANGUAGE:

- The 8051 microcontroller has only one data type in assembly language programming. It is 8 bits, and the size of each register is also 8 bits.
- The data types used by the 8051 can be positive or negative.

DIRECTIVES (PSEUDOCODES):/ASSEMBLER DIRECTIVE

Directives or assembler directives are instructions given to the assembler. The following are directives used in 8051

DB (Define Byte):

- It is used to define the 8-bit data.
- When DB is used to define data, the numbers can be in decimal, binary, hex, or ASCII formats.

Example: DATA1 DB 24 ; 1 byte of memory is assigned for variable DATA1 and set its value with decimal 24

DATA2 DB 34H ; 1 byte of memory is assigned for variable DATA2 and set its value with hex value 34

EQU (Equate):

- This is used to define a constant without occupying a memory location
Example:

```
COUNT    EQU    25
...
MOV      R3, #COUNT
```

When executing the instruction "MOV R3, #COUNT", the register R3 will be loaded with the value 25.

ORG (Origin):

- The ORG directive is used to indicate the beginning of the address.
- The number that comes after ORG can be either in hex or in decimal. If the number is not followed by H, it is decimal and the assembler will convert it to hex.

END:

- Another important DIRECTIVE is the END directive.
- This indicates to the assembler the end of the source (asm) file.
- The END directive is the last line of an 8051 program, meaning that in the source code anything after the END directive is ignored by the assembler.

4.4 DELAY ROUTINE

- Ø In an 8051 microcontroller, it requires 12 T-states for executing a single instruction cycle.
- Ø For an 8051 microcontroller clocked by a 12MHz crystal, the time taken for executing one instruction cycle is 1μs and it is according to the equation, *Time for 1 instruction cycle = 12 / 12MHz = 1μs*.
- Ø The shortest instructions will execute in 1μs and instructions like DJNZ Rx, LABEL will take 2 μs.

Delay routine with 500μs delay:

```

        MOV R6, #250D          ;delay program with 500μs delay
LABEL1: DJNZ R6, LABEL1
        RET

```

- Ø The above program roughly produces a delay of 500μs.
- Ø The instruction DJNZ Rx, LABEL is a two cycle instruction and it will take 2μs to execute.
- Ø So repeating this instruction 250 times will generate a delay of 250 x 2μs = 500μs.

Delay routine with 1mS delay:

```

DELAY:  MOV R6, #250D
        MOV R7, #250D
LABEL1: DJNZ R6, LABEL1
LABEL2: DJNZ R7, LABEL2
        RET

```

Delay routine with 1S delay:

- Ø The program shown below produces a delay of around 1 second.
- Ø In this program subroutine for delaying 1mS (DELAY) is called 4 times back to back and the entire cycle is repeated 250 times. As result, a delay of 4 x 1mS x 250 = 1000mS = 1 second is produced.

```

DELAY:  MOV R5, #250D LABEL: ACALL
DELAY1  ACALL DELAY1
        ACALL DELAY1
        ACALL DELAY1
        DJNZ R5, LABEL RET
DELAY1: MOV R6, #250D MOV R7, #250D
LOOP1:  DJNZ R6, LOOP1
LOOP2:  DJNZ R7, LOOP2
        RET

```

4.5 I/O PORT PROGRAMMING

- Ø There are four ports P0, P1, P2 and P3 each use 8 pins, making them 8-bit ports.
- Ø All the ports upon RESET are configured as output, ready to be used as output ports.
- Ø To use any of these ports as an input port, it must be programmed.

Port 0:

- Ø Port 0 occupies a total of 8 pins (pins 32-39) .It can be used for input or output.
- Ø To use the pins of port 0 as both input and output ports, each pin must be connected externally to a 10K ohm pull-up resistor.

Ktunotes.in

To program Port0 as output port:

- Ø Upon reset, port 0 is configured as an output port.
- Ø For example, the following code will continuously send out to port 0 the alternating values 00H and FFH

```
        MOV A,#00H
BACK:   MOV P0,A
        ACALL DELAY
        CPL A
        SJMP BACK
```

```
DELAY:  MOV R6,#250D
        MOV R7,#250D
LABEL1: DJNZ R6,LABEL1
LABEL2: DJNZ R7,LABEL2
        RET
```

To program Port0 as input port:

- Ø In order to make it an input, the port must be programmed by writing 1 to all the bits.
- Ø In the following code, port 0 is configured first as an input port by writing 1's to it, and then data is received from the port and sent to P1.

```
        MOV A,#0FFH    ; A = FF hex
        MOV P0,A       ; initialize P0 as an input port
BACK:   MOV A,P0        ;get data from P0
        MOV P1,A       ;send it to port 1
        SJMP BACK
```

Port 1:

- Ø Port 1 occupies a total of 8 pins (pins 1 through 8). It can be used as input or output.
- Ø This port does not need any pull-up resistors since it already has pull-up resistors internally.

To program Port1 as output port:

- Ø Upon reset, Port 1 is configured as an output port.
- Ø For example, the following code will continuously send out to port1 the alternating values 00h & FFh

```
        MOV A,#00H      ; A = 00 hex
BACK:   MOV P1,A        ;send it to Port 1

        ACALL DELAY     ;call delay routine
        CPL A
        SJMP BACK
```

To program Port1 as input port:

- Ø To make port1 an input port, it must be programmed as such by writing 1 to all its bits.
- Ø In the following code port1 is configured first as an input port by writing 1's to it, then data is received from the port and saved in R7.

```

MOV A,#0FFH ;A=FF HEX
MOV P1,A ;initialize P1 as an input port by writing all 1's to it
MOV A,P1 ;get data from P1
MOV R7,A ;save it in register R7

```

Port 2 :

- Ø Port 2 occupies a total of 8 pins (pins 21- 28). It can be used as input or output.
- Ø P2 does not need any pull-up resistors since it already has pull-up resistors internally.

To program Port2 as output port:

- Ø Upon reset, Port 2 is configured as an output port.
- Ø For example, the following code will send out continuously to port 2 the alternating values 00h and FFh.

```

                MOV A,#00H ; A = 00 hex
BACK: MOV P2,A ;send it to Port 2
                ACALL DELAY ;call delay routine
                CPL A
                SJMP BACK

```

To program Port2 as input port:

- Ø To make port2 an input port, it must be programmed as such by writing 1 to all its bits.
- Ø In the following code port2 is configured first as an input port by writing 1's to it, then data is received from the port and saved in R7.

```

MOV A,#0FFH ;A=FF HEX
MOV P2,A ;initialize P2 as an input port by writing all 1's to it
MOV A,P2 ;get data from P2
MOV R7,A ;save it in register R7

```

Port 3 :

- Ø Port 3 occupies a total of 8 pins, pins 10 through 17. It can be used as input or output.
- Ø P3 does not need any pull-up resistors.
- Ø Port 3 is configured as an output port upon reset.

Program for generating 1KHz square wave.

Frequency=1KHz. Therefore, $T=1/1\text{KHz} = 1\text{mS}$. Therefore, ON Time= 500 μS and OFF Time=500 μS

```

                ORG 000H
                MOV A, #00h
MAIN  MOV P1,A
                ACALL DELAY
                CPL A
                SJMP MAIN
                END

DELAY:  MOV R6,#250D ;delay program with 500  $\mu\text{S}$  delay
LABEL1: DJNZ R6,LABEL1
                RET

```


4.6 DATAT YPES IN C

1. Unsigned char

- Ø Since the 8051 is an 8-bit microcontroller, the character data type is the most natural choice for many applications.
- Ø The unsigned char is an 8-bit data type that takes a value in the range of 0 – 255 (00 – FFH).

Write an 8051 C program to send values 00 - FF to port P1.

Solution:

```
#include <reg51.h>
void main(void)
{
    unsigned char z;
    for(z=0; z<=255; z++)
        P1=z;
}
```

Run the above program on your simulator to see how P1 displays values 00 - FFH in binary.

2. Signed char

- Ø The signed char is an 8-bit data type that uses the most significant bit (D7 of D7 – D0) to represent the – or + value.
- Ø As a result, we have only 7 bits for the magnitude of the signed number, giving us values from -128 to +127.
- Ø Notice that if we do not use the keyword *unsigned*, the default is the signed value.

Write an 8051 C program to send values of -4 to +4 to port P1.

Solution:

```
//sign numbers
#include <reg51.h>
void main(void)
{
    char mynum[] = {+1, -1, +2, -2, +3, -3, +4, -4};
    unsigned char z;
    for(z=0; z<=8; z++)
        P1=mynum [z];
}
```

3. Unsigned int

- Ø The unsigned int is a 16-bit data type that takes a value in the range of 0 to 65535 (0000 – FFFFH).
- Ø In the 8051, unsigned int is used to define 16-bit variables such as memory addresses.

4. Signed int

- Ø Signed int is a 16-bit data type that uses the most significant bit (D15 of D15 – D0) to represent the – or + value.
- Ø As a result, we have only 15 bits for the magnitude of the number, or values from -32,768 to +32,767.

5. Sbit (single bit)

- Ø The sbit keyword is a widely used 8051 C data type designed specifically to access single-bit addressable registers.
- Ø It allows access to the single bits of the SFR registers.

Ex:- Write an 8051 C program to toggle bit DO of the port PI (P1.O) 50,000 times.

Solution:

```
#include <reg51.h>
sbit MYBIT = P1.0;    //notice that sbit is
                      //declared outside of main

void main(void)
{
    unsigned int z;
    for (z=0; z<=50000; z++)
    {
        MYBIT = 0;
        MYBIT = 1;
    }
}
```

6. Bit

- Ø The bit data type allows access to single bits of bit-addressable memory spaces 20 – 2FH.

7. Sfr

- Ø To access the full byte of SFR registers, we use the sfr data type.

4.7 Time Delay in C

- Ø 8051 has two different ways to generate time delay using C programming.
- Ø The **first method** is simply using **Loop** program function in which *Delay()* function is made or by providing *for()*; delay loop in Embedded C programming.
- Ø For example- *for(i=0;i<"any decimal value";i++);* this is the delay for loop used in embedded C.

Code to generate 250 ms delay on Port P1 of 8051:

Program: Write an 8051C program toggle bits of Port1 continuously with 250mS delay.

```
#include "REG51.h"
void MSDelay(unsigned int);
void main(void)
{
    while (1) //repeat forever
    {
        P1=0x55;    // To make alternate bits of port1 0 and 1(0101 0101)
        MSDelay(250);
        P1=0xAA;    // To invert bits of port1. (1010 1010)
        MSDelay(250);
    }
}
```

```

void MSDelay(unsigned int itime)
{
    unsigned int i,j;
    for (i=0;i<itime;i++) //this is For loop delay used to define delay value in C
        for (j=0;j<1275;j++);
}

```

4.8 I/O port programming in C

Byte size I/O

1. LEDs are connected to bits P1 and P2. Write an 8051 C program that shows the count from 0 to FFH (0000 0000 to 1111 1111 in binary) on the LEDs.

Solution:

```

#include <reg51.h>
#define LED P2          //notice how we can define P2
void main(void)
{
    P1=00;              //clear P1
    LED=0;              //clear P2
    for(;;)              //repeat forever
    {
        P1++;           //increment P1
        LED++;          //increment P2
    }
}

```

2. Write an 8051 C program to get a byte of data from P1, wait 1/2 second, and then send it to P2.

Solution:

```

#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
{
    unsigned char mybyte;
    P1=0xFF;          //make P1 an input port
    while(1)
    {
        mybyte=P1;    //get a byte from P1
        MSDelay(500);
        P2=mybyte;    //send it to P2
    }
}

void MSDelay(unsigned int itime)
{
    unsigned int i, j;
    for i=0;i<itime;i++)
        for(j=0;j<1275;j++);
}

```

Bit-addressable I/O programming

1. Write an 8051 C program to toggle only bit P2.4 continuously without disturbing the rest of the bits of P2.

Solution:

```
//toggling an individual bit
#include <reg51.h>
sbit mybit = P2^4;    //notice the way single bit is declared
void main(void)
{
    while(1)
    {
        mybit=1;        //turn on P2.4
        mybit=0;        //turn off P2.4
    }
}
```

2. Write an 8051 C program to monitor bit P1.5. If it is high, send 55H to P0; otherwise, send AAH to P2.

Solution:

```
#include <reg51.h>
sbit mybit = P1^5;    //notice the way single bit is declared
void main(void)
{
    mybit=1;           //make mybit an input
    while(1)
    {
        if(mybit==1)
            P0=0x55;
        else
            P2=0xAA;
    }
}
```

